

```
quizA |= 1UL << 27;           // 指出第 27 个学生通过了测验
status = quizA & (1UL << 27); // 检查第 27 个学生是否通过了测验
quizA &= ~(1UL << 27);       // 第 27 个学生未通过测验
// 使用标准库类 bitset 完成等价的工作
bitset<30> quizB;             // 每个学生分配一位, 所有位都被初始化为 0
quizB.set(27);               // 指出第 27 个学生通过了测验
status = quizB[27];          // 检查第 27 个学生是否通过了测验
quizB.reset(27);             // 第 27 个学生未通过测验
```

17.2.2 节练习

练习 17.10: 使用序列 1、2、3、5、8、13、21 初始化一个 bitset，将这些位置置位。对另一个 bitset 进行默认初始化，并编写一小段程序将其恰当的位置位。

练习 17.11: 定义一个数据结构，包含一个整型对象，记录一个包含 10 个问题的真/假测验的解答。如果测验包含 100 道题，你需要对数据结构做出什么改变(如果需要的话)?

练习 17.12: 使用前一题中的数据结构，编写一个函数，它接受一个问题编号和一个表示真/假解答的值，函数根据这两个参数更新测验的解答。

练习 17.13: 编写一个整型对象，包含真/假测验的正确答案。使用它来为前两题中的数据结构生成测验成绩。

17.3 正则表达式

正则表达式 (regular expression) 是一种描述字符序列的方法，是一种极其强大的计算工具。但是，用于定义正则表达式的描述语言已经大大超出了本书的范围。因此，我们重点介绍如何使用 C++ 正则表达式库 (RE 库)，它是新标准库的一部分。RE 库定义在头文件 `regex` 中，它包含多个组件，列于表 17.4 中。

C++
11

表 17.4: 正则表达式库组件	
<code>regex</code>	表示有一个正则表达式的类
<code>regex_match</code>	将一个字符序列与一个正则表达式匹配
<code>regex_search</code>	寻找第一个与正则表达式匹配的子序列
<code>regex_replace</code>	使用给定格式替换一个正则表达式
<code>sregex_iterator</code>	迭代器适配器，调用 <code>regex_search</code> 来遍历一个 <code>string</code> 中所有匹配的子串
<code>smatch</code>	容器类，保存在 <code>string</code> 中搜索的结果
<code>ssub_match</code>	<code>string</code> 中匹配的子表达式的结果



如果你还不熟悉正则表达式的使用，你应该浏览这一节，以获得正则表达式可以做什么的一些概念。

`regex` 类表示一个正则表达式。除了初始化和赋值之外，`regex` 还支持其他一些操作。表 17.6 (第 647 页) 列出了 `regex` 支持的操作。

函数 `regex_match` 和 `regex_search` 确定一个给定字符序列与一个给定 `regex`

是否匹配。如果整个输入序列与表达式匹配，则 `regex_match` 函数返回 `true`；如果输入序列中一个子串与表达式匹配，则 `regex_search` 函数返回 `true`。还有一个 `regex_replace` 函数，我们将在 17.3.4 节（第 657 页）中介绍。

表 17.5 列出了 `regex` 的函数的参数。这些函数都返回 `bool` 值，且都被重载了：其中一个版本接受一个类型为 `smatch` 的附加参数。如果匹配成功，这些函数将成功匹配的相关信息保存在给定的 `smatch` 对象中。

表 17.5: <code>regex_search</code> 和 <code>regex_match</code> 的参数	
注意：这些操作返回 <code>bool</code> 值，指出是否找到匹配。	
<code>(seq, m, r, mft)</code> <code>(seq, r, mft)</code>	在字符序列 <code>seq</code> 中查找 <code>regex</code> 对象 <code>r</code> 中的正则表达式。 <code>seq</code> 可以是一个 <code>string</code> 、表示范围的一对迭代器以及一个指向空字符结尾的字符数组的指针 <code>m</code> 是一个 <code>match</code> 对象，用来保存匹配结果的相关细节。 <code>m</code> 和 <code>seq</code> 必须具有兼容的类型（参见 17.3.1 节，第 649 页） <code>mft</code> 是一个可选的 <code>regex_constants::match_flag_type</code> 值。表 17.13（第 659 页）描述了这些值，它们会影响匹配过程

17.3.1 使用正则表达式库

我们从一个非常简单的例子开始——查找违反众所周知的拼写规则“i 除非在 c 之后，否则必须在 e 之前”的单词：

```
// 查找不在字符 c 之后的字符串 ei
string pattern("[^c]ei");
// 我们需要包含 pattern 的整个单词
pattern = "[[:alpha:]]*" + pattern + "[[:alpha:]]*";
regex r(pattern);      // 构造一个用于查找模式的 regex
smatch results;        // 定义一个对象保存搜索结果
// 定义一个 string 保存与模式匹配和不匹配的文本
string test_str = "receipt freind theif receive";
// 用 r 在 test_str 中查找与 pattern 匹配的子串
if (regex_search(test_str, results, r))    // 如果有匹配子串
    cout << results.str() << endl;        // 打印匹配的单词
```

我们首先定义了一个 `string` 来保存希望查找的正则表达式。正则表达式 `[^c]` 表明我们希望匹配任意不是 ‘c’ 的字符，而 `[^c]ei` 指出我们想要匹配这种字符后接 `ei` 的字符串。此模式描述的字符串恰好包含三个字符。我们想要包含此模式的单词的完整内容。为了与整个单词匹配，我们还需要一个正则表达式与这个三字母模式之前和之后的字母匹配。

730

这个正则表达式包含零个或多个字母后接我们的三字母的模式，然后再接零个或多个额外的字母。默认情况下，`regex` 使用的正则表达式语言是 ECMAScript。在 ECMAScript 中，模式 `[[:alpha:]]` 匹配任意字母，符号 `+` 和 `*` 分别表示我们希望“一个或多个”或“零个或多个”匹配。因此 `[[:alpha:]]*` 将匹配零个或多个字母。

将正则表达式存入 `pattern` 后，我们用它来初始化一个名为 `r` 的 `regex` 对象。接下来我们定义了一个 `string`，用来测试正则表达式。我们将 `test_str` 初始化为与模式匹配的单词（如“freind”和“thief”）和不匹配的单词（如“recepit”和“receive”）。我们还定义了一个名为 `results` 的 `smatch` 对象，它将被传递给 `regex_search`。如果找到匹配子串，`results` 将会保存匹配位置的细节信息。

接下来我们调用了 `regex_search`。如果它找到匹配子串，就返回 `true`。我们用 `results` 的 `str` 成员来打印 `test_str` 中与模式匹配的部分。函数 `regex_search` 在输入序列中只要找到一个匹配子串就会停止查找。因此，程序的输出将是

`freind`

17.3.2 节（第 650 页）将会介绍如何查找输入序列中所有的匹配子串。

指定 `regex` 对象的选项

当我们定义一个 `regex` 或是对一个 `regex` 调用 `assign` 为其赋予新值时，可以指定一些标志来影响 `regex` 如何操作。这些标志控制 `regex` 对象的处理过程。表 17.6 列出的最后 6 个标志指出编写正则表达式所用的语言。对这 6 个标志，我们必须设置其中之一，且只能设置一个。默认情况下，ECMAScript 标志被设置，从而 `regex` 会使用 ECMA-262 规范，这也是很多 Web 浏览器所使用的正则表达式语言。

表 17.6: <code>regex</code> （和 <code>wregex</code> ）选项	
<code>regex r(re)</code>	<code>re</code> 表示一个正则表达式，它可以是一个 <code>string</code> 、一个表示字符范围的迭代器对、一个指向空字符结尾的字符数组的指针、一个字符指针和一个计数器或是一个花括号包围的字符列表。
<code>regex r(re, f)</code>	<code>f</code> 是指出对象如何处理的标志。 <code>f</code> 通过下面列出的值来设置。如果未指定 <code>f</code> ，其默认值为 <code>ECMAScript</code>
<code>r1 = re</code>	将 <code>r1</code> 中的正则表达式替换为 <code>re</code> 。 <code>re</code> 表示一个正则表达式，它可以是另一个 <code>regex</code> 对象、一个 <code>string</code> 、一个指向空字符结尾的字符数组的指针或是一个花括号包围的字符列表
<code>r1.assign(re, f)</code>	与使用赋值运算符(=)效果相同；可选的标志 <code>f</code> 也与 <code>regex</code> 的构造函数中对应的参数含义相同
<code>r.mark_count()</code>	<code>r</code> 中子表达式的数目（我们将在 17.3.3 节（第 654 页）中介绍）
<code>r.flags()</code>	返回 <code>r</code> 的标志集

注：构造函数和赋值操作可能抛出类型为 `regex_error` 的异常。

定义 <code>regex</code> 时指定的标志	
定义在 <code>regex</code> 和 <code>regex_constants::syntax_option_type</code> 中	
<code>icase</code>	在匹配过程中忽略大小写
<code>nosubs</code>	不保存匹配的子表达式
<code>optimize</code>	执行速度优先于构造速度
<code>ECMAScript</code>	使用 ECMA-262 指定的语法
<code>basic</code>	使用 POSIX 基本的正则表达式语法
<code>extended</code>	使用 POSIX 扩展的正则表达式语法
<code>awk</code>	使用 POSIX 版本的 <code>awk</code> 语言的语法
<code>grep</code>	使用 POSIX 版本的 <code>grep</code> 的语法
<code>egrep</code>	使用 POSIX 版本的 <code>egrep</code> 的语法

其他 3 个标志允许我们指定正则表达式处理过程中与语言无关的方面。例如，我们可以指出希望正则表达式以大小写无关的方式进行匹配。

作为一个例子，我们可以用 `icase` 标志查找具有特定扩展名的文件名。大多数操作系统都是按大小写无关的方式来识别扩展名的——可以将一个 C++ 程序保存在 `.cc` 结尾的文件中，也可以保存在 `.Cc`、`.cC` 或是 `.CC` 结尾的文件中，效果是一样的。如下所示，我

们可以编写一个正则表达式来识别上述任何一种扩展名以及其他普通文件扩展名：

```
// 一个或多个字母或数字字符后接一个'.'再接"cpp"或"cxx"或"cc"
regex r("[[:alnum:]]+\\. (cpp|cxx|cc)$", regex::icase);
smatch results;
string filename;
while (cin >> filename)
    if (regex_search(filename, results, r))
        cout << results.str() << endl; // 打印匹配结果
```

此表达式将匹配这样的字符串：一个或多个字母或数字后接一个句点再接三个文件扩展名之一。这样，此正则表达式将会匹配指定的文件扩展名而不会理会大小写。

就像 C++ 语言中有特殊字符一样（参见 2.1.3 节，第 36 页），正则表达式语言通常也有特殊字符。例如，字符点 (.) 通常匹配任意字符。与 C++ 一样，我们可以在字符之前放置一个反斜线来去掉其特殊含义。由于反斜线也是 C++ 中的一个特殊字符，我们在字符串字面常量中必须连续使用两个反斜线来告诉 C++ 我们想要一个普通反斜线字符。因此，为了表示与句点字符匹配的正则表达式，必须写成 \.（第一个反斜线去掉 C++ 语言中反斜线的特殊含义，即，正则表达式字符串为 \.，第二个反斜线则表示在正则表达式中去掉的特殊含义）。

732 指定或使用正则表达式时的错误

我们可以将正则表达式本身看作用一种简单程序设计语言编写的“程序”。这种语言不是由 C++ 编译器解释的。正则表达式是在运行时，当一个 regex 对象被初始化或被赋予一个新模式时，才被“编译”的。与任何其他程序设计语言一样，我们用这种语言编写的正则表达式也可能有错误。



需要意识到的非常重要的一点是，一个正则表达式的语法是否正确是在运行时解析的。

如果我们编写的正则表达式存在错误，则在运行时标准库会抛出一个类型为 **regex_error** 的异常（参见 5.6 节，第 173 页）。类似标准异常类型，**regex_error** 有一个 **what** 操作来描述发生了什么错误（参见 5.6.2 节，第 175 页）。**regex_error** 还有一个名为 **code** 的成员，用来返回某个错误类型对应的数值编码。**code** 返回的值是由具体实现定义的。RE 库能抛出的标准错误如表 17.7 所示。

例如，我们可能在模式中意外遇到一个方括号：

```
try {
    // 错误：alnum 漏掉了右括号，构造函数会抛出异常
    regex r("[[:alnum:]]+\\. (cpp|cxx|cc)$", regex::icase);
} catch (regex_error e)
{ cout << e.what() << "\n code: " << e.code() << endl; }
```

当这段程序在我们的系统上运行时，程序会生成：

```
regex_error(error_brack):
The expression contained mismatched [ and ].
code: 4
```

表 17.7: 正则表达式错误类型	
定义在 <code>regex</code> 和 <code>regex_constants::error_type</code> 中	
<code>error_collate</code>	无效的元素校对请求
<code>error_ctype</code>	无效的字符类
<code>error_escape</code>	无效的转义字符或无效的尾置转义
<code>error_backref</code>	无效的向后引用
<code>error_brack</code>	不匹配的方括号 (<code>[或]</code>)
<code>error_paren</code>	不匹配的小括号 (<code>(或)</code>)
<code>error_brace</code>	不匹配的花括号 (<code>{或}</code>)
<code>error_badbrace</code>	<code>{}</code> 中无效的范围
<code>error_range</code>	无效的字符范围 (如 <code>[z-a]</code>)
<code>error_space</code>	内存不足, 无法处理此正则表达式
<code>error_badrepeat</code>	重复字符 (<code>*</code> 、 <code>?</code> 、 <code>+</code> 或 <code>{}</code>) 之前没有有效的正则表达式
<code>error_complexity</code>	要求的匹配过于复杂
<code>error_stack</code>	栈空间不足, 无法处理匹配

我们的编译器定义了 `code` 成员, 返回表 17.7 列出的错误类型的编号, 与往常一样, 编号从 0 开始。 733

建议：避免创建不必要的正则表达式

如我们所见, 一个正则表达式所表示的“程序”是在运行时而非编译时编译的。正则表达式的编译是一个非常慢的操作, 特别是在你使用了扩展的正则表达式语法或是复杂的正则表达式时。因此, 构造一个 `regex` 对象以及向一个已存在的 `regex` 赋予一个新的正则表达式可能是非常耗时的。为了最小化这种开销, 你应该努力避免创建很多不必要的 `regex`。特别是, 如果你在一个循环中使用正则表达式, 应该在循环外创建它, 而不是在每步迭代时都编译它。

正则表达式类和输入序列类型

我们可以搜索多种类型的输入序列。输入可以是普通 `char` 数据或 `wchar_t` 数据, 字符可以保存在标准库 `string` 中或是 `char` 数组中 (或是宽字符版本, `wstring` 或 `wchar_t` 数组中)。RE 为这些不同的输入序列类型都定义了对应的类型。

例如, `regex` 类保存类型 `char` 的正则表达式。标准库还定义了一个 `wregex` 类保存类型 `wchar_t`, 其操作与 `regex` 完全相同。两者唯一的差别是 `wregex` 的初始值必须使用 `wchar_t` 而不是 `char`。

匹配和迭代器类型 (我们将在下面小节中介绍) 更为特殊。这些类型的差异不仅在于字符类型, 还在于序列是在标准库 `string` 中还是在数组中: `smatch` 表示 `string` 类型的输入序列; `cmatch` 表示字符数组序列; `wsmatch` 表示宽字符串 (`wstring`) 输入; 而 `wcmatch` 表示宽字符数组。

重点在于我们使用的 RE 库类型必须与输入序列类型匹配。表 17.8 指出了 RE 库类型与输入序列类型的对应关系。例如:

```
regex r("[[:alnum:]]+\\. (cpp|cxx|cc)$", regex::icase);
smatch results; // 将匹配 string 输入序列, 而不是 char*
if (regex_search("myfile.cc", results, r)) // 错误: 输入为 char*
    cout << results.str() << endl;
```

734 这段代码会编译失败, 因为 `match` 参数的类型与输入序列的类型不匹配。如果我们希望搜索一个字符数组, 就必须使用 `cmatch` 对象:

```
cmatch results; // 将匹配字符数组输入序列
if (regex_search("myfile.cc", results, r))
    cout << results.str() << endl; // 打印当前匹配
```

本书程序一般会使用 `string` 输入序列和对应的 `string` 版本的 RE 库组件。

表 17.8: 正则表达式库类	
如果输入序列类型	则使用正则表达式类
<code>string</code>	<code>regex</code> 、 <code>smatch</code> 、 <code>ssub_match</code> 和 <code>sregex_iterator</code>
<code>const char*</code>	<code>regex</code> 、 <code>cmatch</code> 、 <code>csub_match</code> 和 <code>cregex_iterator</code>
<code>wstring</code>	<code>wregex</code> 、 <code>wsmatch</code> 、 <code>wssub_match</code> 和 <code>wsregex_iterator</code>
<code>const wchar_t*</code>	<code>wregex</code> 、 <code>wcmatch</code> 、 <code>wcsub_match</code> 和 <code>wcregex_iterator</code>

17.3.1 节练习

- 练习 17.14: 编写几个正则表达式, 分别触发不同错误。运行你的程序, 观察编译器对每个错误的输出。
- 练习 17.15: 编写程序, 使用模式查找违反“i 在 e 之前, 除非在 c 之后”规则的单词。你的程序应该提示用户输入一个单词, 然后指出此单词是否符合要求。用一些违反和未违反规则的单词测试你的程序。
- 练习 17.16: 如果前一题程序中的 `regex` 对象用“`^[^c]ei`”进行初始化, 将会发生什么? 用此模式测试你的程序, 检查你的答案是否正确。

17.3.2 匹配与 Regex 迭代器类型

第 646 页中的程序查找违反“i 在 e 之前, 除非在 c 之后”规则的单词, 它只打印输入序列中第一个匹配的单词。我们可以使用 `sregex_iterator` 来获得所有匹配。`regex` 迭代器是一种迭代器适配器(参见 9.6 节, 第 329 页), 被绑定到一个输入序列和一个 `regex` 对象上。如表 17.8 所述, 每种不同输入序列类型都有对应的特殊 `regex` 迭代器类型。迭代器操作如表 17.9 所述。

表 17.9: <code>sregex_iterator</code> 操作	
这些操作也适用于 <code>cregex_iterator</code> 、 <code>wsregex_iterator</code> 和 <code>wcregex_iterator</code> 。	
<code>sregex_iterator</code>	一个 <code>sregex_iterator</code> , 遍历迭代器 <code>b</code> 和 <code>e</code> 表示的 <code>string</code> 。
<code>it(b, e, r);</code>	它调用 <code>sregex_search(b, e, r)</code> 将 <code>it</code> 定位到输入中第一个匹配的位置

续表

sregex_iterator	sregex_iterator 的尾后迭代器
end;	
*it	根据最后一个调用 regex_search 的结果, 返回一个 smatch 对象的引用或一个指向 smatch 对象的指针
it->	
++it	从输入序列当前匹配位置开始调用 regex_search。前置版本返回递增后迭代器; 后置版本返回旧值
it++	
it1 == it2	如果两个 sregex_iterator 都是尾后迭代器, 则它们相等
it1 != it2	如果两个 sregex_iterator 都是非尾后迭代器, 则它们相等

当我们将一个 sregex_iterator 绑定到一个 string 和一个 regex 对象时, 迭代器自动定位到给定 string 中第一个匹配位置。即, sregex_iterator 构造函数对给定 string 和 regex 调用 regex_search。当我们解引用迭代器时, 会得到一个对应最近一次搜索结果的 smatch 对象。当我们递增迭代器时, 它调用 regex_search 在输入 string 中查找下一个匹配。

使用 sregex_iterator

作为一个例子, 我们将扩展之前的程序, 在一个文本文件中查找所有违反“i 在 e 之前, 除非在 c 之后”规则的单词。我们假定名为 file 的 string 保存了我们要搜索的输入文件的全部内容。这个版本的程序将使用与前一个版本一样的 pattern, 但会使用一个 sregex_iterator 来进行搜索:

735

```
// 查找前一个字符不是 c 的字符串 ei
string pattern("[^c]ei");
// 我们想要包含 pattern 的单词的全部内容
pattern = "[[:alpha:]]*" + pattern + "[[:alpha:]]*";
regex r(pattern, regex::icase); // 在进行匹配时将忽略大小写
// 它将反复调用 regex_search 来寻找文件中的所有匹配
for (sregex_iterator it(file.begin(), file.end(), r), end_it;
     it != end_it; ++it)
    cout << it->str() << endl; // 匹配的单词
```

for 循环遍历 file 中每个与 r 匹配的子串。for 语句中的初始值定义了 it 和 end_it。当我们定义 it 时, sregex_iterator 的构造函数调用 regex_search 将 it 定位到 file 中第一个与 r 匹配的位置。而 end_it 是一个空 sregex_iterator, 起到尾后迭代器的作用。for 语句中的递增运算通过 regex_search 来“推进”迭代器。当我们解引用迭代器时, 会得到一个表示当前匹配结果的 smatch 对象。我们调用它的 str 成员来打印匹配的单词。

我们可以将此循环想象为不断从一个匹配位置跳到下一个匹配位置, 如图 17.1 所示。

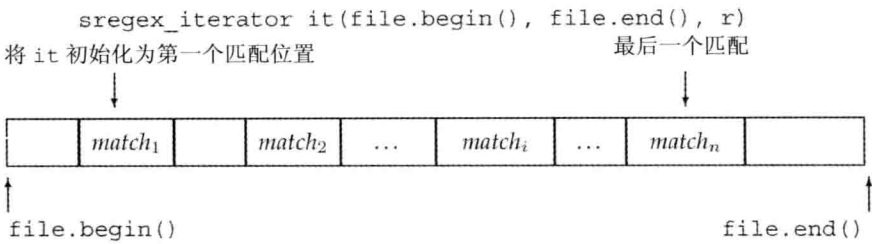


图 17.1: 使用 sregex_iterator

使用匹配数据

如果我们对最初版本程序中的 test_str 运行此循环，则输出将是

```
freind
theif
```

但是，仅获得与我们的正则表达式匹配的单词还不是那么有用。如果我们在一个更大的输入序列——例如，在本章英文版的文本上运行此程序——可能希望看到匹配单词出现的上下文，如

```
hey read or write according to the type
>>> being <<<
handled. The input operators ignore whi
```

除了允许打印输入字符串中匹配的部分之外，匹配结果类还提供了有关匹配结果的更多细节信息。表 17.10 和表 17.11 列出了这些类型支持的操作。

736

我们将在下一节中介绍更多有关 smatch 和 ssub_match 类型的内容。目前，我们只需知道它们允许我们获得匹配的上下文即可。匹配类型有两个名为 prefix 和 suffix 的成员，分别返回表示输入序列中当前匹配之前和之后部分的 ssub_match 对象。一个 ssub_match 对象有两个名为 str 和 length 的成员，分别返回匹配的 string 和该 string 的大小。我们可以用这些操作重写语法程序的循环。

```
// 循环头与之前一样
for (sregex_iterator it(file.begin(), file.end(), r), end_it;
     it != end_it; ++it) {
    auto pos = it->prefix().length();           // 前缀的大小
    pos = pos > 40 ? pos - 40 : 0;              // 我们想要最多 40 个字符
    cout << it->prefix().str().substr(pos)      // 前缀的最后一部分
         << "\n\t\t\t>>> " << it->str() << " <<<\n" // 匹配的单词
         << it->suffix().str().substr(0, 40)    // 后缀的第一部分
         << endl;
}
```

循环本身的工作方式与前一个程序相同。改变的是循环内部，如图 17.2 所示。

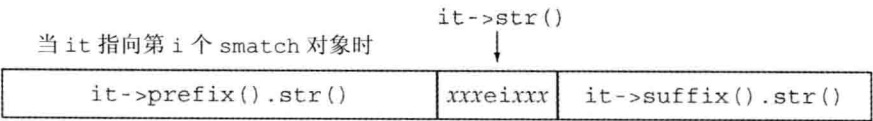


图 17.2: smatch 对象表示一个特定匹配

我们调用 prefix，返回一个 ssub_match 对象，表示 file 中当前匹配之前的部分。

我们对此 `ssub_match` 对象调用 `length`，获得前缀部分的字符数目。接下来调整 `pos`，使之指向前缀部分末尾向前 40 个字符的位置。如果前缀部分的长度小于 40 个字符，我们将 `pos` 置为 0，表示要打印整个前缀部分。我们用 `substr`（参见 9.5.1 节，第 321 页）来打印指定位置到前缀部分末尾的内容。

打印了当前匹配之前的字符之后，我们接下来用特殊格式打印匹配的单词本身，使得它在输出中能突出显示出来。打印匹配单词之后，我们打印 `file` 中匹配部分之后的前（最多）40 个字符。

737

表 17.10: smatch 操作	
这些操作也适用于 <code>cmatch</code> 、 <code>wsmatch</code> 、 <code>wcmatch</code> 和对应的 <code>csub_match</code> 、 <code>wssub_match</code> 和 <code>wcsub_match</code> 。	
<code>m.ready()</code>	如果已经通过调用 <code>regex_serach</code> 或 <code>regex_match</code> 设置了 <code>m</code> ，则返回 <code>true</code> ；否则返回 <code>false</code> 。如果 <code>ready</code> 返回 <code>false</code> ，则对 <code>m</code> 进行操作是未定义的
<code>m.size()</code>	如果匹配失败，则返回 0；否则返回最近一次匹配的正则表达式中子表达式的数目
<code>m.empty()</code>	若 <code>m.size()</code> 为 0，则返回 <code>true</code>
<code>m.prefix()</code>	一个 <code>ssub_match</code> 对象，表示当前匹配之前的序列
<code>m.suffix()</code>	一个 <code>ssub_match</code> 对象，表示当前匹配之后的部分
<code>m.format(...)</code>	见表 17.12（第 657 页）
在接受一个索引的操作中， <code>n</code> 的默认值为 0 且必须小于 <code>m.size()</code> 。	
第一个子匹配（索引为 0）表示整个匹配。	
<code>m.length(n)</code>	第 <code>n</code> 个匹配的子表达式的大小
<code>m.position(n)</code>	第 <code>n</code> 个子表达式距序列开始的距离
<code>m.str(n)</code>	第 <code>n</code> 个子表达式匹配的 <code>string</code>
<code>m[n]</code>	对应第 <code>n</code> 个子表达式的 <code>ssub_match</code> 对象
<code>m.begin(), m.end()</code>	表示 <code>m</code> 中 <code>sub_match</code> 元素范围的迭代器。与往常一样， <code>cbegin</code>
<code>m.cbegin(), m.cend()</code>	和 <code>cend</code> 返回 <code>const_iterator</code>

17.3.2 节练习

练习 17.17: 更新你的程序，令它查找输入序列中所有违反“ei”语法规则的单词。

练习 17.18: 修改你的程序，忽略包含“ei”但并非拼写错误的单词，如“albeit”和“neighbor”。

17.3.3 使用子表达式

738

正则表达式中的模式通常包含一个或多个子表达式（`subexpression`）。一个子表达式是模式的一部分，本身也具有意义。正则表达式语法通常用括号表示子表达式。

例如，我们用来匹配 C++ 文件的模式（参见 17.3.1 节，第 646 页）就是用括号来分组可能的文件扩展名。每当我们用括号分组多个可行选项时，同时也就声明了这些选项形成子表达式。我们可以重写扩展名表达式，以使得模式中点之前表示文件名的部分也形成子表达式，如下所示：

```
// r 有两个子表达式：第一个是点之前表示文件名的部分，第二个表示文件扩展名
regex r("([[:alnum:]]+)\.\.(cpp|cxx|cc)$", regex::icase);
```

现在我们的模式包含两个括号括起来的子表达式：

- `([[:alnum:]]+)`，匹配一个或多个字符的序列
- `(cpp|cxx|cc)`，匹配文件扩展名

我们还可以重写 17.3.1 节（第 646 页）中的程序，通过修改输出语句使之只打印文件名。

```
if (regex_search(filename, results, r))
    cout << results.str(1) << endl; // 打印第一个子表达式
```

与最初的程序一样，我们还是调用 `regex_search` 在名为 `filename` 的 `string` 中查找模式 `r`，并且传递 `smatch` 对象 `results` 来保存匹配结果。如果调用成功，我们打印结果。但是，在此版本中，我们打印的是 `str(1)`，即，与第一个子表达式匹配的部分。

匹配对象除了提供匹配整体的相关信息外，还提供访问模式中每个子表达式的能力。子匹配是按位置来访问的。第一个子匹配位置为 0，表示整个模式对应的匹配，随后是每个子表达式对应的匹配。因此，本例模式中第一个子表达式，即表示文件名的子表达式，其位置为 1，而文件扩展名对应的子表达式位置为 2。

例如，如果文件名为 `foo.cpp`，则 `results.str(0)` 将保存 `foo.cpp`；`results.str(1)` 将保存 `foo`；而 `results.str(2)` 将保存 `cpp`。在此程序中，我们想要点之前的那部分名字，即第一个子表达式，因此我们打印 `results.str(1)`。

子表达式用于数据验证

子表达式的一个常见用途是验证必须匹配特定格式的数据。例如，美国的电话号码有十位数字，包含一个区号和一个七位的本地号码。区号通常放在括号里，但这并不是必需的。剩余七位数字可以用一个短横线、一个点或是一个空格分隔，但也可以完全不用分隔符。我们可能希望接受任何这种格式的数据而拒绝任何其他格式的数。我们将分两步来实现这一目标：首先，我们将用一个正则表达式找到可能是电话号码的序列，然后再调用一个函数来完成数据验证。

在编写电话号码模式之前，我们需要介绍一下 ECMAScript 正则表达式语言的一些特性：

- `\{d\}` 表示单个数字而 `\{d\}{n}` 则表示一个 n 个数字的序列。（如，`\{d\}{3}` 匹配三个数字的序列。）
- 在方括号中的字符集合表示匹配这些字符中任意一个。（如，`[-.]` 匹配一个短横线或一个点或一个空格。注意，点在括号中没有特殊含义。）
- 后接 `'?'` 的组件是可选的。（如，`\{d\}{3}[-.]? \{d\}{4}` 匹配这样的序列：开始是三个数字，后接一个可选的短横线或点或空格，然后是四个数字。此模式可以匹配 `555-0132` 或 `555.0132` 或 `555 0132` 或 `5550132`。）
- 类似 C++，ECMAScript 使用反斜线表示一个字符本身而不是其特殊含义。由于我们的模式包含括号，而括号是 ECMAScript 中的特殊字符，因此我们必须用 `\`（和 `\`）来表示括号是我们的模式的一部分而不是特殊字符。

由于反斜线是 C++ 中的特殊字符，在模式中每次出现 `\` 的地方，我们都必须用一个额外的反斜线来告知 C++ 我们需要一个反斜线字符而不是一个特殊符号。因此，我们用 `\\{d\}{3}` 来表示正则表达式 `\{d\}{3}`。

为了验证电话号码，我们需要访问模式的组成部分。例如，我们希望验证区号部分的数字如果用了左括号，那么它是否也在区号后面用了右括号。即，我们不希望出现 (908.555.1800 这样的号码。

为了获得匹配的组成部分，我们需要在定义正则表达式时使用子表达式。每个子表达式用一对括号包围：

```
// 整个正则表达式包含七个子表达式：( ddd )分隔符 ddd 分隔符 dddd
// 子表达式 1、3、4 和 6 是可选的；2、5 和 7 保存号码
"(\(\)?(\d{3})(\))?([-.]?)(\d{3})([-.]?) (\d{4})";
```

由于我们的模式使用了括号，而且必须去除反斜线的特殊含义，因此这个模式很难读（也很难写！）。理解此模式的最简单的方法是逐个剥离（括号包围的）子表达式：

1. `(\(\)?` 表示区号部分可选的左括号
2. `(\d{3})` 表示区号
3. `(\))?` 表示区号部分可选的右括号
4. `([-.]?)` 表示区号部分可选的分隔符
5. `(\d{3})` 表示号码的下三位数字
6. `([-.]?)` 表示可选的分隔符
7. `(\d{4})` 表示号码的最后四位数字

下面的代码读取一个文件，并用此模式查找与完整的电话号码模式匹配的数据。它会调用一个名为 `valid` 的函数来检查号码格式是否合法：

```
string phone =
    "(\(\)?(\d{3})(\))?([-.]?)(\d{3})([-.]?) (\d{4})";
regex r(phone); // regex 对象，用于查找我们的模式
smatch m;
string s;
// 从输入文件中读取每条记录
while (getline(cin, s)) {
    // 对每个匹配的电话号码
    for (sregex_iterator it(s.begin(), s.end(), r), end_it;
         it != end_it; ++it)
        // 检查号码的格式是否合法
        if (valid(*it))
            cout << "valid: " << it->str() << endl;
        else
            cout << "not valid: " << it->str() << endl;
}
```

使用子匹配操作

我们将使用表 17.11 中描述的子匹配操作来编写 `valid` 函数。需要记住的重要一点是，我们的 `pattern` 有七个子表达式。与往常一样，每个 `smatch` 对象会包含八个 `ssub_match` 元素。位置 [0] 的元素表示整个匹配；元素 [1]...[7] 表示每个对应的子表达式。

当调用 `valid` 时，我们知道已经有一个完整的匹配，但不知道每个可选的子表达式是否是匹配的一部分。如果一个子表达式是完整匹配的一部分，则其对应的 `ssub_match` 对象的 `matched` 成员为 `true`。

表 17.11：子匹配操作

注意：这些操作适用于 <code>ssub_match</code> 、 <code>csub_match</code> 、 <code>wssub_match</code> 、 <code>wcsub_match</code> 。	
<code>matched</code>	一个 <code>public bool</code> 数据成员，指出此 <code>ssub_match</code> 是否匹配了
<code>first</code>	<code>public</code> 数据成员，指向匹配序列首元素和尾后位置的迭代器。如果
<code>second</code>	未匹配，则 <code>first</code> 和 <code>second</code> 是相等的
<code>length()</code>	匹配的大小。如果 <code>matched</code> 为 <code>false</code> ，则返回 0
<code>str()</code>	返回一个包含输入中匹配部分的 <code>string</code> 。如果 <code>matched</code> 为 <code>false</code> ，则返回空 <code>string</code>
<code>s = ssub</code>	将 <code>ssub_match</code> 对象 <code>ssub</code> 转化为 <code>string</code> 对象 <code>s</code> 。等价于 <code>s=ssub.str()</code> 。转换运算符不是 <code>explicit</code> 的（参见 14.9.1 节，第 515 页）

741 在一个合法的电话号码中，区号要么是完整括号包围的，要么完全没有括号。因此，`valid` 要做什么工作依赖于号码是否以一个括号开始：

```
bool valid(const smatch& m)
{
    // 如果区号前有一个左括号
    if(m[1].matched)
        // 则区号后必须有一个右括号，之后紧跟剩余号码或一个空格
        return m[3].matched
            && (m[4].matched == 0 || m[4].str() == " ");
    else
        // 否则，区号后不能有右括号
        // 另两个组成部分间的分隔符必须匹配
        return !m[3].matched
            && m[4].str() == m[6].str();
}
```

我们首先检查第一个子表达式（即，左括号）是否匹配了。这个子表达式在 `m[1]` 中。如果匹配了，则号码是以左括号开始的。在此情况下，如果区号后的子表达式也匹配了（意味着区号后有右括号）则整个号码是合法的。而且，如果号码正确使用了括号，则下一个字符必须是一个空格或下一部分的第一个数字。

如果 `m[1]` 未匹配，（即，没有左括号），则区号后的子表达式也不应该匹配。如果它为`空`，则整个号码是合法的。

17.3.3 节练习

- 练习 17.19: 为什么可以不先检查 `m[4]` 是否匹配了就直接调用 `m[4].str()`？
- 练习 17.20: 编写你自己版本的验证电话号码的程序。
- 练习 17.21: 使用本节中定义的 `valid` 函数重写 8.3.2 节（第 289 页）中的电话号码程序。
- 练习 17.22: 重写你的电话号码程序，使之允许在号码的三个部分之间放置任意多个空白符。
- 练习 17.23: 编写查找邮政编码的正则表达式。一个美国邮政编码可以由五位或九位数字组成。前五位数字和后四位数字之间可以用一个短横线分隔。

17.3.4 使用 `regex_replace`

正则表达式不仅用在我们希望查找一个给定序列的时候，还用在当我们想将找到的序列替换为另一个序列的时候。例如，我们可能希望将美国的电话号码转换为“ddd.ddd.dddd”的形式，即，区号和后面三位数字用一个点分隔。

当我们希望在输入序列中查找并替换一个正则表达式时，可以调用 `regex_replace`。表 17.12 描述了 `regex_replace`，类似搜索函数，它接受一个输入字符串和一个 `regex` 对象，不同的是，它还接受一个描述我们想要的输出形式的字符串。

◀ 742

表 17.12: 正则表达式替换操作	
<code>m.format(dest, fmt, mft)</code> <code>m.format(fmt, mft)</code>	使用格式字符串 <code>fmt</code> 生成格式化输出，匹配在 <code>m</code> 中，可选的 <code>match_flag_type</code> 标志在 <code>mft</code> 中。第一个版本写入迭代器 <code>dest</code> 指向的目的位置（参见 10.5.1 节，第 365 页）并接受 <code>fmt</code> 参数，可以是一个 <code>string</code> ，也可以是表示字符数组中范围的一对指针。第二个版本返回一个 <code>string</code> ，保存输出，并接受 <code>fmt</code> 参数，可以是一个 <code>string</code> ，也可以是一个指向空字符结尾的字符数组的指针。 <code>mft</code> 的默认值为 <code>format_default</code>
<code>regex_replace(dest, seq, r, fmt, mft)</code> <code>regex_replace(seq, r, fmt, mft)</code>	遍历 <code>seq</code> ，用 <code>regex_search</code> 查找与 <code>regex</code> 对象 <code>r</code> 匹配的子串。使用格式字符串 <code>fmt</code> 和可选的 <code>match_flag_type</code> 标志来生成输出。第一个版本将输出写入到迭代器 <code>dest</code> 指定的位置，并接受一对迭代器 <code>seq</code> 表示范围。第二个版本返回一个 <code>string</code> ，保存输出，且 <code>seq</code> 既可以是一个 <code>string</code> 也可以是一个指向空字符结尾的字符数组的指针。在所有情况下， <code>fmt</code> 既可以是一个 <code>string</code> 也可以是一个指向空字符结尾的字符数组的指针，且 <code>mft</code> 的默认值为 <code>match_default</code>

替换字符串由我们想要的字符组合与匹配的子串对应的子表达式而组成。在本例中，我们希望在替换字符串中使用第二个、第五个和第七个子表达式。而忽略第一个、第三个、第四个和第六个子表达式，因为这些子表达式用来形成号码的原格式而非新格式中的一部分。我们用一个符号 `$` 后跟子表达式的索引号来表示一个特定的子表达式：

```
string fmt = "$2.$5.$7"; // 将号码格式改为 ddd.ddd.dddd
```

可以像下面这样使用我们的正则表达式模式和替换字符串：

```
regex r(phone); // 用来寻找模式的 regex 对象
string number = "(908) 555-1800";
cout << regex_replace(number, r, fmt) << endl;
```

此程序的输出为：

```
908.555.1800
```

只替换输入序列的一部分

正则表达式更有趣的一个用处是替换一个大文件中的电话号码。例如，我们有一个保存人名及其电话号码的文件：

```
743 > morgan (201) 555-2368 862-555-0123
      drew (973)555.0130
      lee (609) 555-0132 2015550175 800.555-0000
```

我们希望将数据转换为下面这样：

```
morgan 201.555.2368 862.555.0123
drew 973.555.0130
lee 609.555.0132 201.555.0175 800.555.0000
```

可以用下面的程序完成这种转换：

```
int main()
{
    string phone =
        "(\\()?(\\d{3}) (\\))?(\\-\\. )?(\\d{3}) (\\-\\. )?(\\d{4})";
    regex r(phone); // 寻找模式所用的 regex 对象
    smatch m;
    string s;
    string fmt = "$2.$5.$7"; // 将号码格式改为 ddd.ddd.dddd
    // 从输入文件中读取每条记录
    while (getline(cin, s))
        cout << regex_replace(s, r, fmt) << endl;
    return 0;
}
```

我们读取每条记录，保存到 `s` 中，并将其传递给 `regex_replace`。此函数在输入序列中查找并转换所有匹配子串。

用来控制匹配和格式的标记

就像标准库定义标志来指导如何处理正则表达式一样，标准库还定义了用来在替换过程中控制匹配或格式的标记。表 17.13 列出了这些值。这些标志可以传递给函数 `regex_search` 或 `regex_match` 或是类 `smatch` 的 `format` 成员。

匹配和格式化标记的类型为 `match_flag_type`。这些值都定义在名为 `regex_constants` 的命名空间中。类似用于 `bind` 的 `placeholders`（参见 10.3.4 节，第 355 页），`regex_constants` 也是定义在命名空间 `std` 中的命名空间。为了使用 `regex_constants` 中的名字，我们必须在名字前同时加上两个命名空间的限定符：

```
using std::regex_constants::format_no_copy;
```

此声明指出，如果代码中使用了 `format_no_copy`，则表示我们想要使用命名空间 `std::constants` 中的这个名字。如下所示，我们也可以用另一种形式的 `using` 来代替上面的代码，我们将在 18.2.2 节（第 702 页）中介绍这种形式：

```
using namespace std::regex_constants;
```

表 17.13: 匹配标志

744

定义在 <code>regex_constants::match_flag_type</code> 中	
<code>match_default</code>	等价于 <code>format_default</code>
<code>match_not_bol</code>	不将首字符作为行首处理
<code>match_not_eol</code>	不将尾字符作为行尾处理
<code>match_not_bow</code>	不将首字符作为单词首处理
<code>match_not_eow</code>	不将尾字符作为单词尾处理
<code>match_any</code>	如果存在多于一个匹配, 则可返回任意一个匹配
<code>match_not_null</code>	不匹配任何空序列
<code>match_continuous</code>	匹配必须从输入的首字符开始
<code>match_prev_avail</code>	输入序列包含第一个匹配之前的内容
<code>format_default</code>	用 ECMAScript 规则替换字符串
<code>format_sed</code>	用 POSIX sed 规则替换字符串
<code>format_no_copy</code>	不输出输入序列中未匹配的部分
<code>format_first_only</code>	只替换子表达式的第一次出现

使用格式标志

默认情况下, `regex_replace` 输出整个输入序列。未与正则表达式匹配的部分会原样输出; 匹配的部分按格式字符串指定的格式输出。我们可以通过在 `regex_replace` 调用中指定 `format_no_copy` 来改变这种默认行为:

```
// 只生成电话号码: 使用新的格式字符串
string fmt2 = "$2.$5.$7 "; // 在最后一部分号码后放置空格作为分隔符
// 通知 regex_replace 只拷贝它替换的文本
cout << regex_replace(s, r, fmt2, format_no_copy) << endl;
```

给定相同的输入, 此版本的程序生成

```
201.555.2368 862.555.0123
973.555.0130
609.555.0132 201.555.0175 800.555.0000
```

17.3.4 节练习

- 练习 17.24: 编写你自己版本的重排电话号码格式的程序。
- 练习 17.25: 重写你的电话号码程序, 使之只输出每个人的第一个电话号码。
- 练习 17.26: 重写你的电话号码程序, 使之对多于一个电话号码的人只输出第二个和后续电话号码。
- 练习 17.27: 编写程序, 将九位数字邮政编码的格式转换为 dddd-dddd。

17.4 随机数

745

程序通常需要一个随机数源。在新标准出现之前, C 和 C++ 都依赖于一个简单的 C 库函数 `rand` 来生成随机数。此函数生成均匀分布的伪随机整数, 每个随机数的范围在 0 和一个系统相关的最大值 (至少为 32767) 之间。

C++ 11